Classical theories of information science

信息选讲之二

Data Compression

涂仕奎
上海交通大学
tushikui@sjtu.edu.cn
https://www.cs.sjtu.edu.cn/~tushikui/

Outline

1. Examples of Codes
2. Kraft Inequality
3. Huffman Codes

1. Examples of Codes

Source code, expected length

Definition A *source code* C for a random variable X is a mapping from $\mathcal{X}$, the range of X, to $\mathcal{D}^*$, the set of finite-length strings of symbols from a D-ary alphabet. Let $C(x)$ denote the codeword corresponding to x and let $l(x)$ denote the length of $C(x)$.

For example, $C(\text{red}) = 00$, $C(\text{blue}) = 11$ is a source code for $\mathcal{X} = \{\text{red}, \text{blue}\}$ with alphabet $\mathcal{D} = \{0, 1\}$.

Definition The *expected length* $L(C)$ of a source code $C(x)$ for a random variable X with probability mass function $p(x)$ is given by

$$L(C) = \sum_{x \in \mathcal{X}} p(x) l(x), \tag{5.1}$$

where $l(x)$ is the length of the codeword associated with x.

Without loss of generality, we can assume that the D-ary alphabet is $\mathcal{D} = \{0, 1, \ldots, D-1\}$.

Example 1.1

Example 5.1.1 Let X be a random variable with the following distribution and codeword assignment:

$$\begin{aligned}
&\Pr(X=1)=\tfrac{1}{2}, && \text{codeword } C(1)=0\\
&\Pr(X=2)=\tfrac{1}{4}, && \text{codeword } C(2)=10\\
&\Pr(X=3)=\tfrac{1}{8}, && \text{codeword } C(3)=110\\
&\Pr(X=4)=\tfrac{1}{8}, && \text{codeword } C(4)=111.
\end{aligned} \tag{5.2}$$

The entropy $H(X)$ of X is 1.75 bits, and the expected length $L(C) = El(X)$ of this code is also 1.75 bits. Here we have a code that has the same average length as the entropy. We note that any sequence of bits can be uniquely decoded into a sequence of symbols of X. For example, the bit string 0110111100110 is decoded as 134213.

Example 1.2

Example 5.1.2 Consider another simple example of a code for a random variable:

$$\begin{aligned} \Pr(X=1) &= \tfrac{1}{3}, \quad \text{codeword } C(1) = 0 \\ \Pr(X=2) &= \tfrac{1}{3}, \quad \text{codeword } C(2) = 10 \\ \Pr(X=3) &= \tfrac{1}{3}, \quad \text{codeword } C(3) = 11. \end{aligned} \tag{5.3}$$

Just as in Example 5.1.1, the code is uniquely decodable. However, in this case the entropy is $\log 3 = 1.58$ bits and the average length of the encoding is 1.66 bits. Here $El(X) > H(X)$.

Example 1.3: Morse code

Example 5.1.3 (*Morse code*) The Morse code is a reasonably efficient code for the English alphabet using an alphabet of four symbols: a dot, a dash, a letter space, and a word space. Short sequences represent frequent letters (e.g., a single dot represents E) and long sequences represent infrequent letters (e.g., Q is represented by "dash,dash,dot,dash"). This is not the optimal representation for the alphabet in four symbols—in fact, many possible codewords are not utilized because the codewords for letters do not contain spaces except for a letter space at the end of every codeword, and no space can follow another space. It is an interesting problem to calculate the number of sequences that can be constructed under these constraints. The problem was solved by Shannon in his original 1948 paper. The problem is also related to coding for magnetic recording, where long strings of 0's are prohibited [5], [370].

Nonsingular code

We now define increasingly more stringent conditions on codes. Let x^n denote $(x_1, x_2, \ldots, x_n)$.

Definition A code is said to be *nonsingular* if every element of the range of X maps into a different string in $\mathcal{D}^*$; that is,

$$x \neq x' \Rightarrow C(x) \neq C(x'). \tag{5.4}$$

Nonsingularity suffices for an unambiguous description of a single value of X. But we usually wish to send a sequence of values of X. In such cases we can ensure decodability by adding a special symbol (a "comma") between any two codewords. But this is an inefficient use of the special symbol; we can do better by developing the idea of self- punctuating or instantaneous codes.

Extension

Definition The *extension* C^* of a code C is the mapping from finite-length strings of $\mathcal{X}$ to finite-length strings of $\mathcal{D}$, defined by

$$C(x_1x_2\cdots x_n) = C(x_1)C(x_2)\cdots C(x_n), \tag{5.5}$$

where $C(x_1)C(x_2)\cdots C(x_n)$ indicates concatenation of the corresponding codewords.

Example 5.1.4 If $C(x_1) = 00$ and $C(x_2) = 11$, then $C(x_1x_2) = 0011$.

Definition A code is called *uniquely decodable* if its extension is non-singular.

In other words, any encoded string in a uniquely decodable code has only one possible source string producing it. However, one may have to look at the entire string to determine even the first symbol in the corresponding source string.

Prefix code

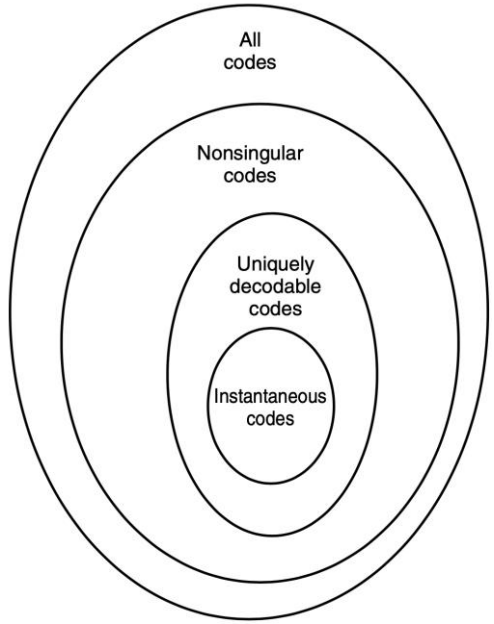
Definition A code is called a *prefix code* or an *instantaneous code* if no codeword is a prefix of any other codeword.

- An *instantaneous code* can be decoded without reference to future code-words since the end of a codeword is immediately recognizable.
- Hence, for an instantaneous code, the symbol x_i can be decoded as soon as we come to the end of the codeword corresponding to it. We need not wait to see the codewords that come later.
- An instantaneous code is a *self-punctuating code*. We can look down the sequence of code symbols and add the commas to separate the codewords without looking at later symbols.
- For example, the binary string 01011111010 produced by the code of Example 5.1.1 is parsed as 0,10,111,110,10.

Classes of Codes

TABLE 5.1 Classes of Codes

X	Singular	Nonsingular, But Not Uniquely Decodable	Uniquely Decodable, But Not Instantaneous	Instantaneous
1	0	0	10	0
2	0	010	00	10
3	0	01	11	110
4	0	10	110	111



2. Kraft Inequality

Kraft Inequality

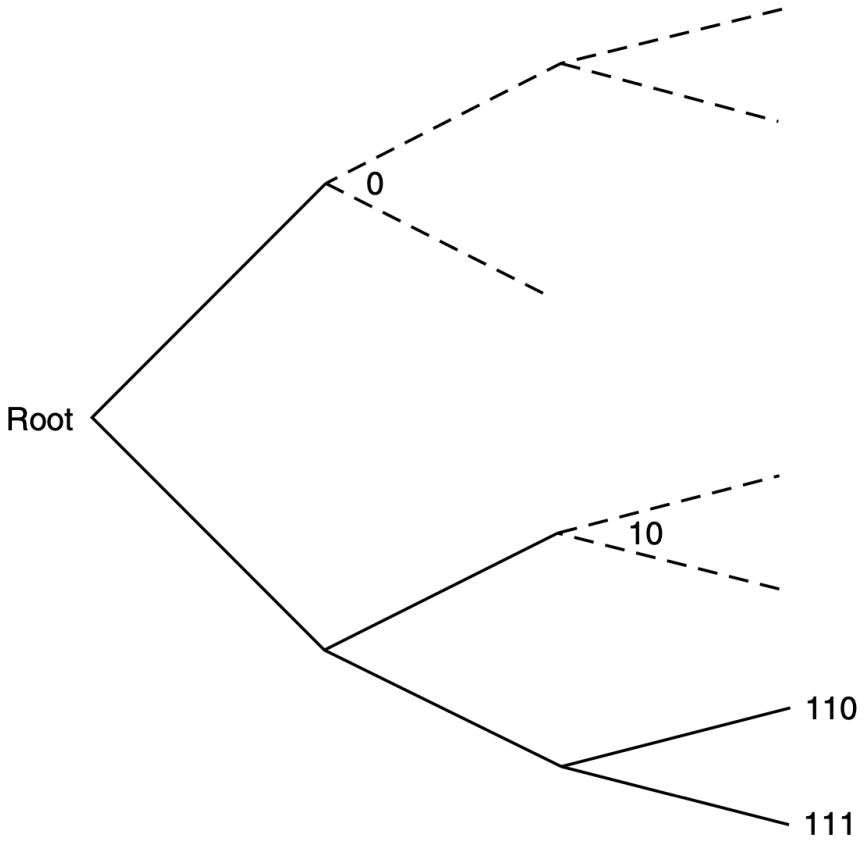
Theorem 5.2.1 (*Kraft inequality*) *For any instantaneous code (prefix code) over an alphabet of size D, the codeword lengths* $l_1, l_2, \ldots, l_m$ *must satisfy the inequality*

$$\sum_i D^{-l_i} \leq 1. \tag{5.6}$$

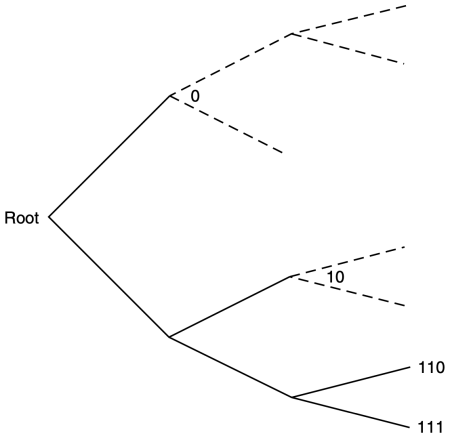
Conversely, given a set of codeword lengths that satisfy this inequality, there exists an instantaneous code with these word lengths.

Proof of Kraft inequality

- Consider a D-ary tree in which each node has D children. Let the branches of the tree represent the symbols of the codeword.
- For example, the D branches arising from the root node represent the D possible values of the first symbol of the codeword. Then **each codeword is represented by a leaf on the tree.** The path from the root traces out the symbols of the codeword.
- A binary example of such a tree is shown in the right Figure. **The prefix condition on the codewords implies that no codeword is an ancestor of any other codeword on the tree.** Hence, each codeword eliminates its descendants as possible codewords.



Proof of Kraft inequality

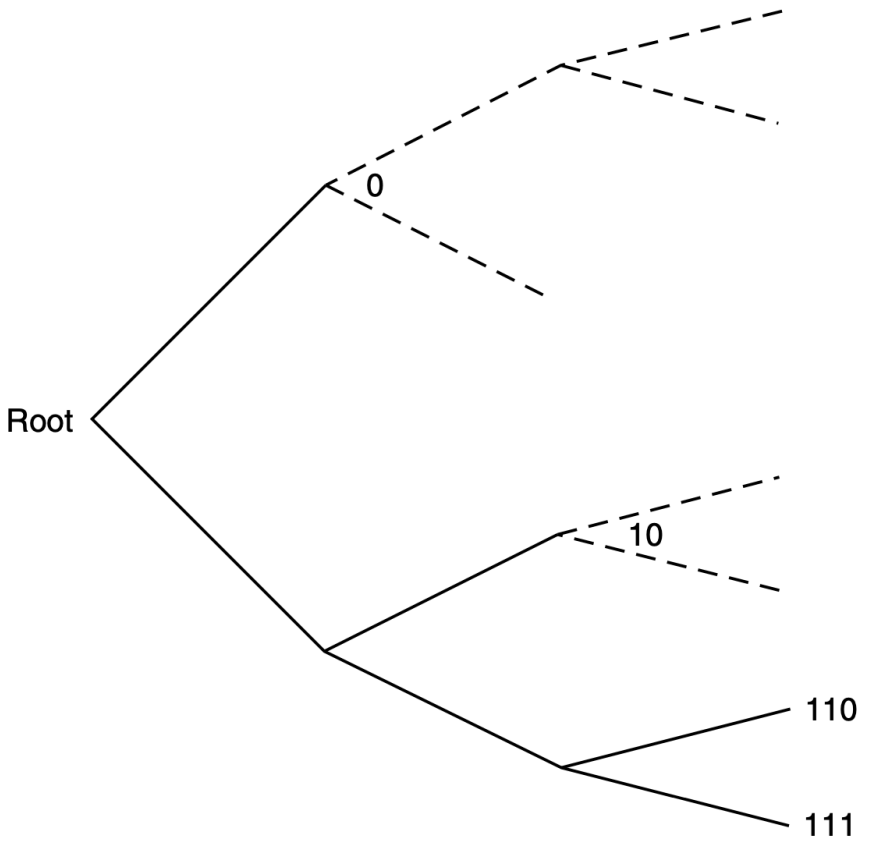


- Let $l_{\max}$ be the length of the longest codeword of the set of codewords.
- Consider all nodes of the tree at level $l_{\max}$. Some of them are codewords, some are descendants of codewords, and some are neither.
- A codeword at level l_i has $D^{l_{\max}-l_i}$ descendants at level $l_{\max}$. Each of these descendant sets must be disjoint. Also, the total number of nodes in these sets must be less than or equal to $D^{l_{\max}}$.
- Hence, summing over all the codewords, we obtain the Kraft inequality as follows:

$$\sum D^{l_{\max}-l_i} \leq D^{l_{\max}} \qquad \text{or} \qquad \sum D^{-l_i} \leq 1$$

Proof of Kraft inequality (Converse part)

- Conversely, given any set of codeword lengths $l_1, l_2, \ldots, l_m$ that satisfy the Kraft inequality, we can always construct a tree like the one in the following figure.
- Label the first node (lexicographically) of depth l_1 as codeword 1, and remove its descendants from the tree.
- Then label the first remaining node of depth l_2 as codeword 2, and so on.
- Proceeding this way, we construct a prefix code with the specified $l_1, l_2, \ldots, l_m$.



Extended Kraft Inequality for infinite prefix code

Theorem 5.2.2 (*Extended Kraft Inequality*) *For any countably infinite set of codewords that form a prefix code, the codeword lengths satisfy the extended Kraft inequality,*

$$\sum_{i=1}^{\infty} D^{-l_i} \leq 1. \tag{5.9}$$

Conversely, given any $l_1, l_2, \ldots$ satisfying the extended Kraft inequality, we can construct a prefix code with these codeword lengths.

Optimal Codes

- We have proved that any codeword set that satisfies the prefix condition has to satisfy the Kraft inequality and that the Kraft inequality is a sufficient condition for the existence of a codeword set with the specified set of codeword lengths.
- We now consider the problem of **finding the prefix code with the minimum expected length.**
- This is equivalent to finding the set of lengths $l_1, l_2, \ldots, l_m$ satisfying the Kraft inequality and whose expected length $L = \sum p_i l_i$ is less than the expected length of any other prefix code.

$$\text{minimize} \quad L = \sum p_i l_i$$

$$\text{subject to} \quad \sum D^{-l_i} \leq 1$$

Optimal Codes

$$\begin{aligned}\text{minimize} \quad & L = \sum p_i l_i \\ \text{subject to} \quad & \sum D^{-l_i} \leq 1\end{aligned}$$

- Use Lagrange multipliers to solve the constrained optimization:

$$J = \sum p_i l_i + \lambda \left(\sum D^{-l_i} \right).$$

Differentiating with respect to l_i, we obtain

$$\frac{\partial J}{\partial l_i} = p_i - \lambda D^{-l_i} \log_e D.$$

Setting the derivative to 0, we obtain

$$D^{-l_i} = \frac{p_i}{\lambda \log_e D}.$$

Substituting this in the constraint to find λ, we find $\lambda = 1/\log_e D$, and hence

$$p_i = D^{-l_i}, \tag{5.18}$$

yielding optimal code lengths,

$$l_i^* = -\log_D p_i. \tag{5.19}$$

This noninteger choice of codeword lengths yields expected codeword length

$$L^* = \sum p_i l_i^* = -\sum p_i \log_D p_i = H_D(X). \tag{5.20}$$

The lower bound of the expected length

Theorem 5.3.1 *The expected length L of any instantaneous D-ary code for a random variable X is greater than or equal to the entropy $H_D(X)$; that is,*

$$L \geq H_D(X), \tag{5.21}$$

with equality if and only if $D^{-l_i} = p_i$.

Proof $L \geq H_D(X)$

Proof: We can write the difference between the expected length and the entropy as

$$L - H_D(X) = \sum p_i l_i - \sum p_i \log_D \frac{1}{p_i} \tag{5.22}$$

$$= -\sum p_i \log_D D^{-l_i} + \sum p_i \log_D p_i. \tag{5.23}$$

Letting $r_i = D^{-l_i} / \sum_j D^{-l_j}$ and $c = \sum D^{-l_i}$, we obtain

$$L - H = \sum p_i \log_D \frac{p_i}{r_i} - \log_D c \tag{5.24}$$

$$= D(\mathbf{p}||\mathbf{r}) + \log_D \frac{1}{c} \tag{5.25}$$

$$\geq 0 \tag{5.26}$$

by the nonnegativity of relative entropy and the fact (Kraft inequality) that $c \leq 1$. Hence, $L \geq H$ with equality if and only if $p_i = D^{-l_i}$ (i.e., if and only if $-\log_D p_i$ is an integer for all i). □

Bounds on the optimal code length

The choice of word lengths $l_i = \log_D \frac{1}{p_i}$ yields $L = H$. Since $\log_D \frac{1}{p_i}$ may not equal an integer, we round it up to give integer word-length assignments,

$$L - H_D = D(\mathbf{p}||\mathbf{r}) - \log\left(\sum D^{-l_i}\right) \geq 0. \tag{5.28}$$

$$l_i = \left\lceil \log_D \frac{1}{p_i} \right\rceil, \tag{5.29}$$

where $\lceil x \rceil$ is the smallest integer $\geq x$. These lengths satisfy the Kraft inequality since

$$\sum D^{-\lceil \log \frac{1}{p_i} \rceil} \leq \sum D^{-\log \frac{1}{p_i}} = \sum p_i = 1. \tag{5.30}$$

This choice of codeword lengths satisfies

$$\log_D \frac{1}{p_i} \leq l_i < \log_D \frac{1}{p_i} + 1. \tag{5.31}$$

Multiplying by p_i and summing over i, we obtain

$$H_D(X) \leq L < H_D(X) + 1. \tag{5.32}$$

Bounds on the optimal code length

Since an optimal code can only be better than this code, we have the following theorem.

Theorem 5.4.1 *Let $l_1^*, l_2^*, \ldots, l_m^*$ be optimal codeword lengths for a source distribution* $\mathbf{p}$ *and a D-ary alphabet, and let L^* be the associated expected length of an optimal code ($L^* = \sum p_i l_i^*$). Then*

$$H_D(X) \leq L^* < H_D(X) + 1. \tag{5.33}$$

Proof: Let $l_i = \lceil \log_D \frac{1}{p_i} \rceil$. Then l_i satisfies the Kraft inequality and from (5.32) we have

$$H_D(X) \leq L = \sum p_i l_i < H_D(X) + 1. \tag{5.34}$$

But since L^*, the expected length of the optimal code, is less than $L = \sum p_i l_i$, and since $L^* \geq H_D$ from Theorem 5.3.1, we have the theorem. □

Over a sequence of symbols

$$H_D(X) \le L^* < H_D(X) + 1.$$

- In Theorem 5.4.1 there is an overhead that is at most 1 bit, due to the fact that $\log(1/p_i)$ is not always an integer.
- We can reduce the overhead per symbol by spreading it out over many symbols.
- With this in mind, let us consider a system in which we send **a sequence of *n* symbols** from X. The symbols are assumed to be drawn *i.i.d.* according to $p(x)$. We can consider these n symbols to be a super symbol from the alphabet $\mathcal{X}^n$.
- Define L_n to be the expected codeword length per input symbol, that is, if $l(x_1, x_2, \ldots, x_n)$ is the length of the binary codeword associated with $(x_1, x_2, \ldots, x_n)$, then

$$L_n = \frac{1}{n}\sum p(x_1, x_2, \ldots, x_n) l(x_1, x_2, \ldots, x_n) = \frac{1}{n} El(X_1, X_2, \ldots, X_n)$$

- We can now apply the bounds derived above to the code:

$$H(X_1, X_2, \ldots, X_n) \le El(X_1, X_2, \ldots, X_n) < H(X_1, X_2, \ldots, X_n) + 1. \qquad (5.36)$$

- Since $X_1, X_2, \ldots, X_n$ are i.i.d., $H(X_1, X_2, \ldots, X_n) = \sum H(X_i) = nH(X)$. Dividing (5.36) by n, we get

$$H(X) \le L_n < H(X) + \frac{1}{n}$$

Over a sequence of symbols

We can use the same argument for a sequence of symbols from a stochastic process that is not necessarily i.i.d. In this case, we still have the bound

$$H(X_1, X_2, \ldots, X_n) \leq El(X_1, X_2, \ldots, X_n) < H(X_1, X_2, \ldots, X_n) + 1. \tag{5.38}$$

Dividing by n again and defining L_n to be the expected description length per symbol, we obtain

$$\frac{H(X_1, X_2, \ldots, X_n)}{n} \leq L_n < \frac{H(X_1, X_2, \ldots, X_n)}{n} + \frac{1}{n}. \tag{5.39}$$

If the stochastic process is stationary, then $H(X_1, X_2, \ldots, X_n)/n \rightarrow H(\mathcal{X})$, and the expected description length tends to the entropy rate as $n \rightarrow \infty$.

Theorem

Theorem 5.4.2 *The minimum expected codeword length per symbol satisfies*

$$\frac{H(X_1, X_2, \ldots, X_n)}{n} \leq L_n^* < \frac{H(X_1, X_2, \ldots, X_n)}{n} + \frac{1}{n}. \tag{5.40}$$

Moreover, if $X_1, X_2, \ldots, X_n$ *is a stationary stochastic process,*

$$L_n^* \to H(\mathcal{X}), \tag{5.41}$$

where $H(\mathcal{X})$ *is the entropy rate of the process.*

This theorem provides another justification for the definition of entropy rate—it is the expected number of bits per symbol required to describe the process.

Expected length under the wrong distribution

- What happens to the expected description length if the code is designed for the **wrong distribution**?
- For example, the wrong distribution may be the best estimate that we can make of the unknown true distribution.

Theorem 5.4.3 *(Wrong code) The expected length under $p(x)$ of the code assignment $l(x) = \left\lceil \log \frac{1}{q(x)} \right\rceil$ satisfies*

$$H(p) + D(p||q) \leq E_p l(X) < H(p) + D(p||q) + 1. \quad (5.42)$$

- What the increase in expected description length due to the incorrect distribution is the relative entropy $D(p||q)$.
- Thus, $D(p||q)$ has a concrete interpretation as the increase in descriptive complexity due to incorrect information.

Proof

Theorem 5.4.3 *(Wrong code) The expected length under $p(x)$ of the code assignment $l(x) = \left\lceil \log \frac{1}{q(x)} \right\rceil$ satisfies*

$$H(p) + D(p||q) \leq E_p l(X) < H(p) + D(p||q) + 1. \tag{5.42}$$

Proof: The expected codelength is

$$El(X) = \sum_x p(x) \left\lceil \log \frac{1}{q(x)} \right\rceil \tag{5.43}$$

$$< \sum_x p(x) \left(\log \frac{1}{q(x)} + 1 \right) \tag{5.44}$$

$$= \sum_x p(x) \log \frac{p(x)}{q(x)} \frac{1}{p(x)} + 1 \tag{5.45}$$

$$= \sum_x p(x) \log \frac{p(x)}{q(x)} + \sum_x p(x) \log \frac{1}{p(x)} + 1 \tag{5.46}$$

$$= D(p||q) + H(p) + 1. \tag{5.47}$$

The lower bound can be derived similarly. □

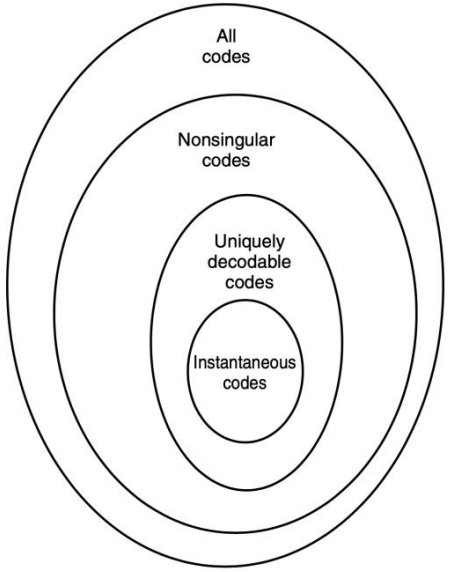
Kraft inequality for uniquely decodable code

Theorem 5.5.1 (*McMillan*) *The codeword lengths of any uniquely decodable D-ary code must satisfy the Kraft inequality*

$$\sum D^{-l_i} \leq 1. \tag{5.48}$$

Conversely, given a set of codeword lengths that satisfy this inequality, it is possible to construct a uniquely decodable code with these codeword lengths.

Corollary *A uniquely decodable code for an infinite source alphabet $\mathcal{X}$ also satisfies the Kraft inequality.*



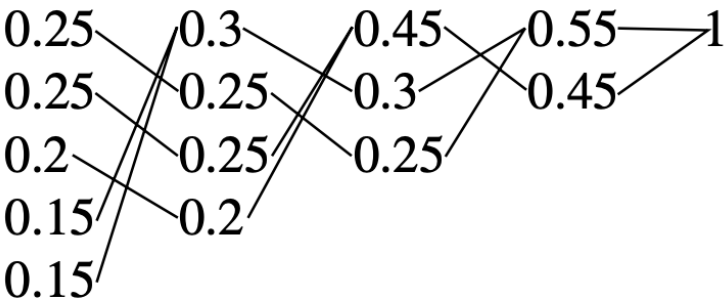
3. Huffman Codes

An optimal (shortest expected length) prefix code for a given distribution can be constructed by a simple algorithm discovered by Huffman

Example (binary code)

- We expect the optimal binary code for X to have the longest codewords assigned to the symbols 4 and 5.
- These two lengths must be equal, since otherwise we can delete a bit from the longer codeword and still have a prefix code, but with a shorter expected length.
- In general, we can construct a code in which the two longest codewords differ only in the last bit.
- For this code, we can combine the symbols 4 and 5 into a single source symbol, with a probability assignment 0.30. Proceeding this way, combining the two least likely symbols into one symbol until we are finally left with only one symbol, and then assigning codewords to the symbols, we obtain the following table:

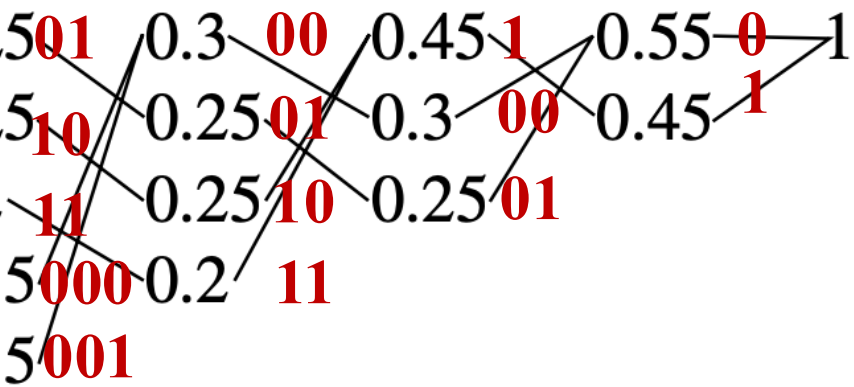
Codeword Length	Codeword	X	Probability
2	01	1	0.25 0.3 0.45 0.55 1
2	10	2	0.25 0.25 0.3 0.45
2	11	3	0.2 0.25 0.25
3	000	4	0.15 0.2
3	001	5	0.15



This code has average length 2.3 bits.

Example (binary code)

Codeword Length	Codeword	X	Probability
2	01	1	0.25 01 0.3 00 0.45 1 0.55 0 1
2	10	2	0.25 10 0.25 01 0.3 00 0.45 1
2	11	3	0.2 11 0.25 10 0.25 01
3	000	4	0.15 000 0.2 11
3	001	5	0.15 001

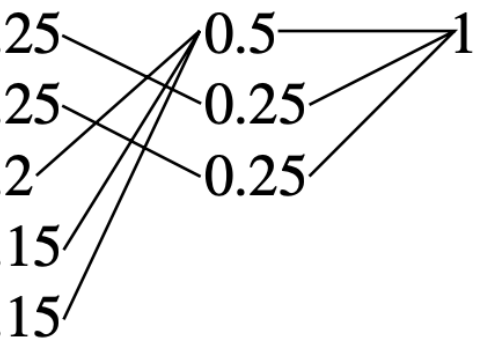


This code has average length 2.3 bits.

Example (ternary code)

Example 5.6.2 Consider a ternary code for the same random variable. Now we combine the three least likely symbols into one supersymbol and obtain the following table:

Codeword	X	Probability		
1	1	0.25	0.5	1
2	2	0.25	0.25	
00	3	0.2	0.25	
01	4	0.15		
02	5	0.15		



This code has an average length of 1.5 ternary digits.

Optimality of Huffman Codes

- We **prove by induction** that the binary Huffman code is optimal.
- There are many optimal codes: inverting all the bits or exchanging two codewords of the same length will give another optimal code. The Huffman procedure constructs one such optimal code.
- To prove the optimality of Huffman codes, we first prove some properties of a particular optimal code.

Without loss of generality, we will assume that the probability masses are ordered, so that $p_1 \geq p_2 \geq \cdots \geq p_m$. Recall that a code is optimal if $\sum p_i l_i$ is minimal.

Properties of the optimal instantaneous code

Lemma 5.8.1 *For any distribution, there exists an optimal instantaneous code (with minimum expected length) that satisfies the following properties:*

1. *The lengths are ordered inversely with the probabilities (i.e., if* $p_j > p_k$*, then* $l_j \leq l_k$*).*
2. *The two longest codewords have the same length.*
3. *Two of the longest codewords differ only in the last bit and correspond to the two least likely symbols.*

Proof of Lemma 5.8.1 (sketch of idea)

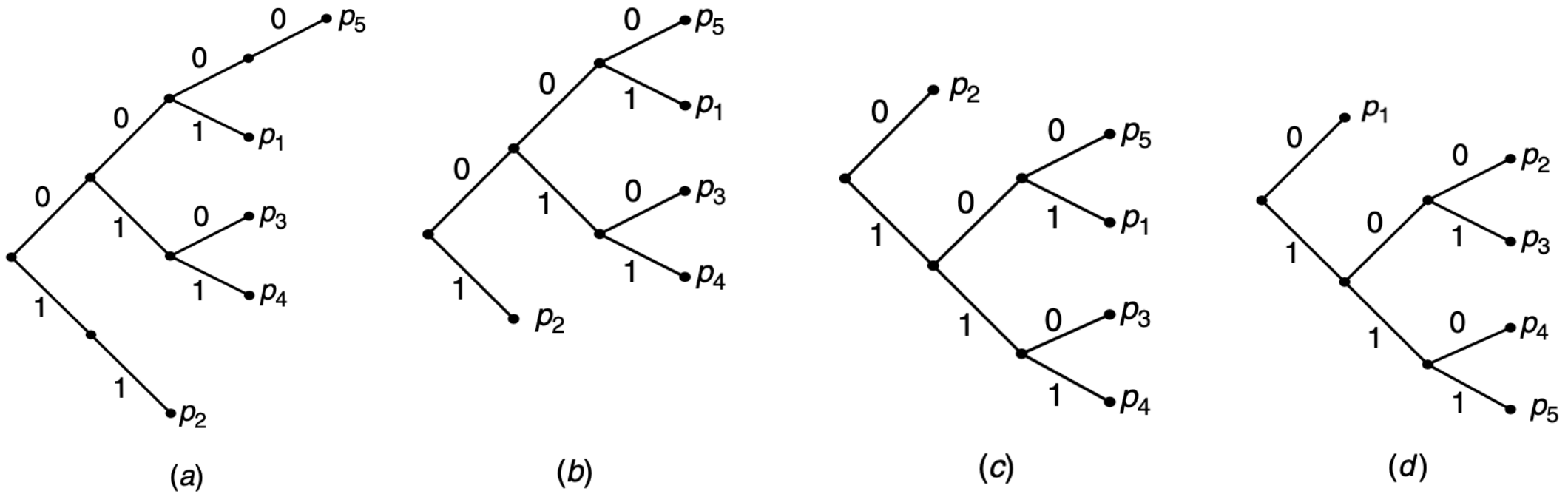


FIGURE 5.3. Properties of optimal codes. We assume that $p_1 \geq p_2 \geq \cdots \geq p_m$. A possible instantaneous code is given in (a). By trimming branches without siblings, we improve the code to (b). We now rearrange the tree as shown in (c), so that the word lengths are ordered by increasing length from top to bottom. Finally, we swap probability assignments to improve the expected depth of the tree, as shown in (d). Every optimal code can be rearranged and swapped into canonical form as in (d), where $l_1 \leq l_2 \leq \cdots \leq l_m$ and $l_{m-1} = l_m$, and the last two codewords differ only in the last bit.

Proof of optimality

Thus, we have shown that there exists an optimal code satisfying the properties of the lemma. We call such codes *canonical* codes. For any probability mass function for an alphabet of size m, $\mathbf{p} = (p_1, p_2, \ldots, p_m)$ with $p_1 \geq p_2 \geq \cdots \geq p_m$, we define the Huffman reduction $\mathbf{p}' = (p_1, p_2, \ldots, p_{m-2}, p_{m-1} + p_m)$ over an alphabet of size $m - 1$ (Figure 5.4). Let $C^*_{m-1}(\mathbf{p}')$ be an optimal code for $\mathbf{p}'$, and let $C^*_m(\mathbf{p})$ be the canonical optimal code for $\mathbf{p}$.

- The proof of optimality will follow from two constructions:
- First, we expand an optimal code for **p**' to construct a code for **p**
- Then we condense an optimal canonical code for **p** to construct a code for the Huffman reduction **p**'.

An illustration on the induction step for Huffman coding

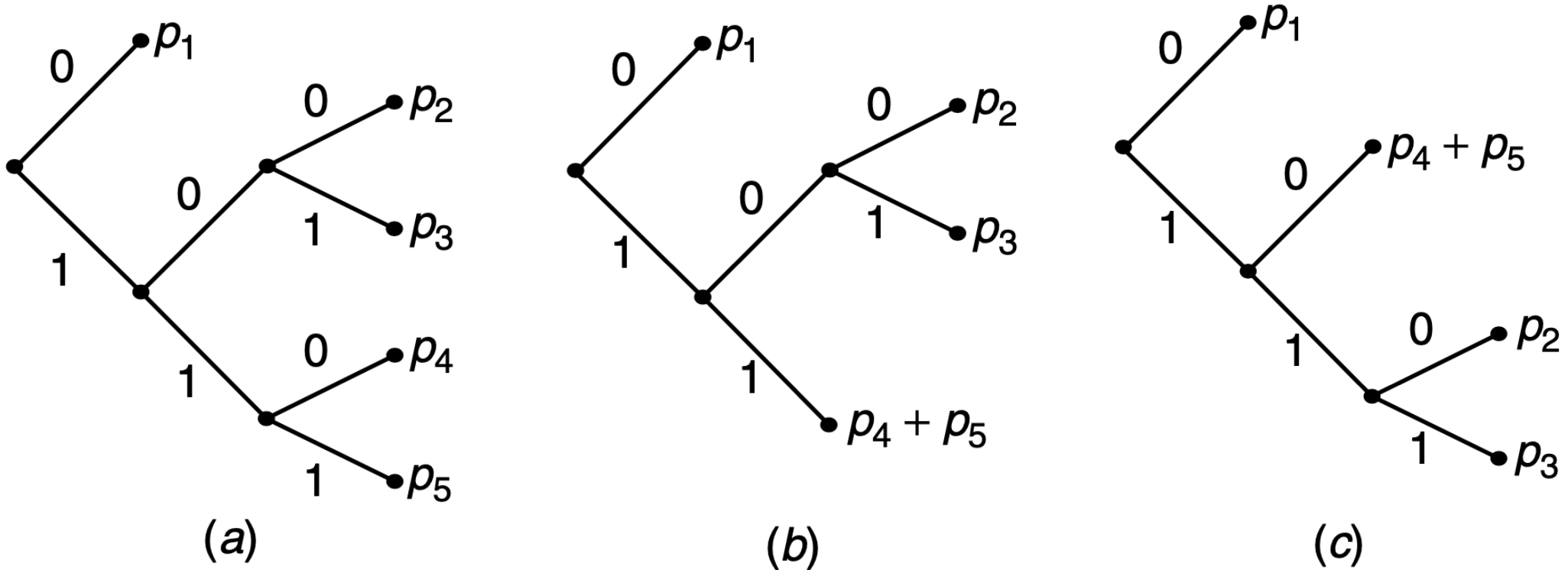


FIGURE 5.4. Induction step for Huffman coding. Let $p_1 \geq p_2 \geq \cdots \geq p_5$. A canonical optimal code is illustrated in (a). Combining the two lowest probabilities, we obtain the code in (b). Rearranging the probabilities in decreasing order, we obtain the canonical code in (c) for $m - 1$ symbols.

Proof of optimality

From the optimal code for $\mathbf{p}'$, we construct an extension code for m elements as follows: Take the codeword in C^*_{m-1} corresponding to weight $p_{m-1} + p_m$ and extend it by adding a 0 to form a codeword for symbol $m-1$ and by adding 1 to form a codeword for symbol m. The code construction is illustrated as follows:

$$
\begin{array}{cccll}
 & C^*_{m-1}(\mathbf{p}') & & C_m(\mathbf{p}) & \\
p_1 & w'_1 & l'_1 & w_1 = w'_1 & l_1 = l'_1 \\
p_2 & w'_2 & l'_2 & w_2 = w'_2 & l_2 = l'_2 \\
\vdots & \vdots & \vdots & \vdots & \vdots \\
p_{m-2} & w'_{m-2} & l'_{m-2} & w_{m-2} = w'_{m-2} & l_{m-2} = l'_{m-2} \\
p_{m-1} + p_m & w'_{m-1} & l'_{m-1} & w_{m-1} = w'_{m-1}0 & l_{m-1} = l'_{m-1} + 1 \\
 & & & w_m = w'_{m-1}1 & l_m = l'_{m-1} + 1
\end{array}
\tag{5.65}
$$

Calculation of the average length $\sum_i p'_i l'_i$ shows that

$$
L(\mathbf{p}) = L^*(\mathbf{p}') + p_{m-1} + p_m. \tag{5.66}
$$

Proof of optimality

Similarly, from the canonical code for $\mathbf{p}$, we construct a code for $\mathbf{p}'$ by merging the codewords for the two lowest-probability symbols $m-1$ and m with probabilities p_{m-1} and p_m, which are siblings by the properties of the canonical code. The new code for $\mathbf{p}'$ has average length

$$L(\mathbf{p}') = \sum_{i=1}^{m-2} p_i l_i + p_{m-1}(l_{m-1}-1) + p_m(l_m - 1) \tag{5.67}$$

$$= \sum_{i=1}^{m} p_i l_i - p_{m-1} - p_m \tag{5.68}$$

$$= L^*(\mathbf{p}) - p_{m-1} - p_m. \tag{5.69}$$

Adding (5.66) and (5.69) together, we obtain

$$L(\mathbf{p}') + L(\mathbf{p}) = L^*(\mathbf{p}') + L^*(\mathbf{p}) \tag{5.70}$$

or

$$(L(\mathbf{p}') - L^*(\mathbf{p}')) + (L(\mathbf{p}) - L^*(\mathbf{p})) = 0. \tag{5.71}$$

Proof of optimality

Now examine the two terms in (5.71). By assumption, since $L^*(\mathbf{p}')$ is the optimal length for $\mathbf{p}'$, we have $L(\mathbf{p}') - L^*(\mathbf{p}') \geq 0$. Similarly, the length of the extension of the optimal code for $\mathbf{p}'$ has to have an average length at least as large as the optimal code for $\mathbf{p}$ [i.e., $L(\mathbf{p}) - L^*(\mathbf{p}) \geq 0$]. But the sum of two nonnegative terms can only be 0 if both of them are 0, which implies that $L(\mathbf{p}) = L^*(\mathbf{p})$ (i.e., the extension of the optimal code for $\mathbf{p}'$ is optimal for $\mathbf{p}$).

Consequently, if we start with an optimal code for $\mathbf{p}'$ with $m - 1$ symbols and construct a code for m symbols by extending the codeword corresponding to $p_{m-1} + p_m$, the new code is also optimal. Starting with a code for two elements, in which case the optimal code is obvious, we can by induction extend this result to prove the following theorem.

Theorem 5.8.1 *Huffman coding is optimal; that is, if C^* is a Huffman code and C' is any other uniquely decodable code, $L(C^*) \leq L(C')$.*

哈夫曼编码的发展历史

- 1951年，哈夫曼和他在MIT信息论的同学需要选择是完成学期报告还是期末考试。导师Robert M. Fano给他们的学期报告的题目是，寻找最有效的二进制编码。由于无法证明哪个已有编码是最有效的，哈夫曼放弃对已有编码的研究，转向新的探索，最终发现了基于有序频率二叉树编码的想法，并很快证明了这个方法是最有效的。由于这个算法，学生终于青出于蓝，超过了他那曾经和信息论创立者香农共同研究过类似编码的导师。哈夫曼使用自底向上的方法构建二叉树，避免了次优算法Shannon-Fano编码的最大弊端——自顶向下构建树。
- 1952年，David A. Huffman在麻省理工攻读博士时发表了《一种构建极小多余编码的方法》（A Method for the Construction of Minimum-Redundancy Codes）一文，它一般就叫做Huffman编码。
- Huffman 在1952年根据香农（Shannon）在1948年和费诺（Fano）在1949年阐述的这种编码思想提出了一种不定长编码的方法，也称霍夫曼（Huffman）编码。霍夫曼编码的基本方法是先对图像数据扫描一遍，计算出各种像素出现的概率，按概率的大小指定不同长度的唯一码字，由此得到一张该图像的霍夫曼码表。编码后的图像数据记录的是每个像素的码字，而码字与实际像素值的对应关系记录在码表中。
- 赫夫曼编码是可变字长编码(VLC)的一种。 Huffman于1952年提出一种编码方法，该方法完全依据字符出现概率来构造异字头的平均长 度最短的码字，有时称之为最佳编码，一般就称Huffman编码。有定理可以保证按字符出现概率分配码长，可使平均码长最短。

哈夫曼编码的应用场景

- 哈夫曼编码（Huffman Coding）是一种广泛使用的数据压缩算法，它通过变长编码表为不同字符分配不同的位数代码，以便于高效的数据存储和传输。以下是哈夫曼编码的几个典型应用场景：
  - **文件压缩**：哈夫曼编码在许多文件压缩工具中被使用，如ZIP和GZIP格式。这些工具利用哈夫曼编码减少文件大小，通过将常用的数据或字符编码为较短的位序列，而不常用的数据则编码为较长的位序列。
  - **多媒体数据压缩**：在JPEG、MP3和MPEG等多媒体数据压缩标准中，哈夫曼编码用于减少图像和音视频数据的大小。例如，JPEG图片压缩首先通过离散余弦变换（DCT）和量化步骤处理图像数据，然后使用哈夫曼编码对这些量化后的值进行编码，以进一步减少数据量。
  - **无线通信**：在无线通信领域，哈夫曼编码可用于优化数据传输，通过压缩数据来减少需要传输的数据量，从而提高传输效率和降低能耗。
  - **网络传输**：在网络数据传输过程中，为了提高传输效率和减少带宽消耗，常常使用哈夫曼编码对数据进行压缩。
  - **文本压缩**：哈夫曼编码也被用于文本文件的压缩，尤其是在文本数据重复率较高的情况下，能显著减少存储空间需求。
  - **编程语言**：一些编程库和框架使用哈夫曼编码来处理内部数据的存储和传输，如某些数据库管理系统中数据的压缩存储。

Thank you!